

BASTION LINE

Rapport Technique Officiel — V1

PWA réellement installable + refonte graphique

Date du rapport	25 septembre 2026
Dépôt	github.com/scuillerjack-oss/bastion-line-v0
Commit de départ (V0 vérifiée)	f4535cef2507f80065cd560f714f309da5c67e9c
Commit final V1 (HEAD distant vérifié)	8d3b9fbc68f239e540b1f1ab99a68d7c0c41b41f
Identifiant de build vérifié	v0.1.0+8d3b9fb (rebuild local GITHUB_SHA=HEAD, confirmé identique)
URL publique	https://scuillerjack-oss.github.io/bastion-line-v0/
Statut	Candidate à la bêta physique V1 — NON VALIDÉE, décision humaine en attente

1. État Git initial constaté (avant toute modification)

Conformément à l'exigence du cahier ("vérifier avant toute modification le HEAD local, le HEAD distant, le commit réellement servi et l'état du workspace"), l'état suivant a été constaté avant d'écrire la moindre ligne de code :

- Workspace : propre, aucune modification non committée (`git status --short` vide).
- HEAD local = `f4535cef2507f80065cd560f714f309da5c67e9c`, branche `main`.
- HEAD distant réel (`git ls-remote`) : identique au HEAD local — aucune divergence.
- Ce commit correspondait à la dernière correction post-bêta V0 (construction mobile, retour visuel base, installabilité PWA), déjà vérifiée déployée avec succès (job `deploy` vert, `pages_build_version` confirmé identique dans le log de déploiement).

Aucune reconstruction du projet, aucun changement de moteur, aucun remplacement d'architecture : le dépôt, sa branche et son historique existants ont été repris tels quels.

2. Priorité absolue — Diagnostic final PWA et solution mise en place

2.1 Diagnostic : pourquoi deux faux positifs précédents

Les cycles précédents avaient corrigé, dans l'ordre : (1) l'absence d'icône raster $\geq 192 \times 192$ requise par les critères d'installabilité Chrome, puis (2) rien d'autre techniquement — le manifest et le service worker étaient déjà corrects. Le diagnostic V1 identifie la cause réelle des faux positifs : **un manifest et un service worker techniquement valides ne garantissent rien si la page est ouverte depuis un navigateur intégré (WebView Android d'une application tierce)**, qui ne propose structurellement aucune option d'installation, quel que soit l'état du code sur cette page.

2.2 Solution mise en place

- Détection fiable d'une WebView Android via le jeton `" ;wv ) "` dans le user-agent — convention documentée volontairement ajoutée par Chromium pour permettre cette détection, pas une heuristique fragile.
- Détection de l'état déjà-installé via `display-mode: standalone`.
- Capture réelle de l'événement natif `beforeinstallprompt` quand Chrome le signale disponible — jamais supposé présent.
- Bannière visible DÈS LE MENU (jamais cachée dans les réglages, jamais bloquante), adaptée à l'état réel détecté :

État détecté	Ce qui est proposé
WebView Android (navigateur intégré)	Message explicite + bouton "Ouvrir dans Chrome" (lien <code>intent://</code> standard)
Invite native disponible (<code>beforeinstallprompt</code>)	Bouton "Installer l'application" déclenchant le vrai prompt Chrome
iOS (Safari ou dérivé)	Instructions manuelles : Partager → Sur l'écran d'accueil
Navigateur normal sans invite native	Instructions génériques : menu du navigateur -> Installer l'application / Ajouter à l'écran d'a
Déjà installée (<code>standalone</code>)	Aucune bannière affichée

Manifest complété : ajout de `id` (identité d'app stable) et `display_override`, deux champs explicitement listés dans la checklist d'audit du cahier. `start_url/scope` (déjà relatifs, donc robustes au sous-chemin `/bastion-line-v0/`) revérifiés par résolution d'URL réelle en test.

2.3 Limite honnête

Cette section ne peut pas être déclarée validée par ce rapport. Conformément au cahier, "la seule validation finale acceptable est une installation réelle et un lancement autonome sur le téléphone physique du porteur". Tout ce qui précède est vérifié par du code et des tests automatisés réels (voir section 4) — pas par une installation physique, qui reste à faire.

3. Priorité 2 — Changements graphiques et techniques

3.1 Principe

Tout est dessiné en Canvas 2D, avec des formes fixes composées (aucune image, aucun sprite, aucune animation image-par-image). Le terrain et le tracé du chemin sont précalculés une seule fois sur un canvas hors-écran puis simplement "blités" chaque frame — coût de rendu inchangé par rapport à la V0.

3.2 Table des changements

Élément	V0	V1
Terrain	Aplat vert uni	Herbe texturée + pierres/buissons décoratifs statiques
Chemin	Trait brun uni	Chemin de terre bordé, avec galets, courbes propres
Base	Cercle marqué "B"	Petite forteresse (créneaux, porte, bannière) + barre de PV visible sur
Emplacement vide	Cercle pointillé + croix	Socle de pierre avec lueur d'invite discrète
Tour rapide	Triangle	Arbalète sur poste (bras anguleux + carreau)
Tour canon	Carré	Tourelle ronde + canon massif
Tour longue portée	Losange	Tour haute effilée (silhouette la plus haute des 4)
Tour contrôle	Hexagone	Tour cylindrique à orbe magique encastré
Ennemis	Ronds de couleur	4 silhouettes : grognard 2-cercles, flèche, bloc riveté, grappe de 3 corp
Projectiles	Formes minimalistes identiques	Différenciés par forme (bille, trait, anneau)

3.3 Auto-critique en cours de production (avant tout commit)

Une capture d'écran + zoom de chaque tour a été prise avant validation. Deux problèmes réels ont été trouvés et corrigés à ce stade, jamais livrés tels quels :

- La première version de la tour "rapide" (arc fin + dôme sombre) se lisait, une fois réduite à l'échelle de jeu, comme la lettre "T" plutôt que comme une arbalète — corrigée avec des bras anguleux flairés et un carreau central.
- La première version de la tour "contrôle" (trapèze + orbe) se lisait comme une silhouette HUMANOÏDE, risquant une confusion directe avec l'ennemi "standard" (lui-même à 2 cercles) — corrigée avec un corps cylindrique à parois droites et des rayons d'énergie autour de l'orbe.



Rendu V1 en jeu : chemin bordé, forteresse, tour "rapide" (arbalète) sélectionnée avec son rayon, ennemis "grognard" sur le chemin.

4. Tests ajoutés/modifiés et résultats complets

4.1 Nouveaux tests

- 5 tests unitaires (`tests/pwaInstall.test.js`) : détection WebView Android réelle vs Chrome Android réel vs Safari iOS vs Chrome desktop ; construction du lien `intent://`.
- 2 tests Playwright (`scripts/check-mobile.mjs`, tests 11 et 12) : bannière "navigateur intégré" affichée avec un vrai user-agent WebView Android simulé au niveau du contexte navigateur ; bouton d'installation natif affiché et déclenchant réellement prompt () quand un événement `beforeinstallprompt` est simulé.
- Extension du test PWA existant (Test 6) : présence du champ `id`, cohérence réelle de `start_url/scope` résolue comme le ferait un navigateur (pas seulement leur présence dans le JSON).

4.2 Résultats complets

Suite	Résultat
Tests unitaires (node:test)	53/53 passants
Vérifications mobiles/PWA (Playwright)	18/18 passantes

L'intégralité des tests V0 existants a été conservée et passe sans modification de leur logique (seuls les tests PWA/mobile ont été étendus, jamais affaiblis).

5. Git, CI/déploiement — vérification réelle, pas supposée

Étape	Résultat vérifié
Commit final	8d3b9fbc68f239e540b1f1ab99a68d7c0c41b41f
HEAD distant après push	Vérifié via <code>git ls-remote</code> — identique au commit local
Job CI "build"	success (53 tests + 18 vérifications mobiles exécutés en CI)
Job CI "deploy"	success — log lu directement, <code>pages_build_version</code> = 8d3b9fbc68f239e540b1f1ab99a68d7c0c41b41f
Build ID re-vérifié localement	v0.1.0+8d3b9fb (rebuild avec <code>GITHUB_SHA=HEAD</code> réel)
URL publique	https://scuillerjack-oss.github.io/bastion-line-v0/

Conformément à l'exigence explicite du cahier ("ne jamais déclarer le déploiement réussi sur la seule base d'une commande lancée ou d'un log partiel"), chaque étape ci-dessus a été vérifiée par une source indépendante (API GitHub pour le HEAD distant et les jobs, log complet du job de déploiement pour le commit réellement servi, rebuild local pour le build ID).

6. Limites restant à valider sur téléphone physique

- **Installation PWA réelle** : la détection WebView/l'invite native/le lien intent:// sont vérifiés par tests automatisés (user-agent et événements simulés), jamais par une installation physique. Le cahier est explicite : ce n'est PAS suffisant pour déclarer cette exigence validée.
 - **Lisibilité graphique en conditions réelles** : les captures d'écran et zooms ont été pris en navigateur headless desktop-simulé-mobile ; la lisibilité finale (contraste, taille perçue, luminosité d'un écran de téléphone en extérieur) reste à confirmer en main.
 - **Performance réelle sur le téléphone du porteur** : le rendu a été conçu pour ne pas ajouter de coût par frame (précalcul terrain/chemin), mais aucune mesure de framerate sur l'appareil physique n'a été effectuée dans cette session.
 - **Chemin "Ouvrir dans Chrome"** : le lien intent:// est un mécanisme standard Android, mais son comportement exact dépend du navigateur intégré précis utilisé par l'app hôte — à confirmer par le porteur dans son contexte réel.
-

7. Conclusion

BASTION LINE V1 reprend strictement le moteur et la boucle de jeu V0 validés par la bêta physique précédente, sans extension de contenu ni rééquilibrage arbitraire. Les deux priorités du cahier ont été traitées : un diagnostic réel (pas une supposition) et une solution technique complète pour l'installabilité PWA, et une refonte graphique pragmatique entièrement en Canvas, elle-même auto-corrigée deux fois après capture d'écran avant tout commit.

Conformément à l'instruction explicite du cahier, BASTION LINE V1 n'est PAS déclarée validée par ce rapport. Ce livrable est une candidate à la bêta physique V1. Seule une installation réelle et un lancement autonome confirmés par le porteur sur son téléphone physique permettront de considérer la priorité absolue comme atteinte ; seule sa propre lecture de l'écran de jeu permettra de juger si la refonte graphique répond réellement au besoin exprimé.